Session 2: NumPy

Programming with Python

Abate Alemayehu

Ethiopian Statistical Service (ESS)

May 2–22, 2026

Overview

Sessions

- Session 1: Introduction to NumPy
- Session 2: NumPy Arrays and Basics
- Session 3: NumPy Array Operations and Broadcasting
- Session 4: NumPy Universal Functions and Aggregations
- Session 5: Analyzing Data with NumPy

Info

NumPy is the foundation of scientific computing in Python.

Slide 86: What is NumPy?

NumPy = Numerical Python

- Fundamental for scientific computing
- Provides multi-dimensional arrays
- Fast mathematical operations
- Written in C for performance

Slide 87: Why NumPy over Python Lists?

Feature	Python List	NumPy Array
Speed	Slow	Fast (C implementation)
Memory	High	Low
Data types	Mixed	Homogeneous
Vectorized ops	No	Yes
Broadcasting	No	Yes

Slide 88: Installing and Importing

Installation

- Included in Anaconda
- `conda install numpy` or `pip install numpy`

Import Convention

```
1 import numpy as np
```

Slide 89: NumPy vs Python Lists – Speed Comparison

Multiplying 1 Million Numbers by 2

Method	Time
Python list (loop)	$\sim$0.1 seconds
NumPy (vectorized)	$\sim$0.005 seconds

Info

NumPy is approximately **20x faster** than Python lists!

Slide 90: The ndarray (n-dimensional array)

Core Data Structure

- **ndarray** = n-dimensional array
- Homogeneous data type (all elements same type)
- Fixed size (cannot append like lists)
- Indexed with integers starting at 0

Slide 91: Array Attributes

```
1 import numpy as np
2 arr = np.array([[1, 2, 3], [4, 5, 6]])
3
4 print(arr.shape)     # (2, 3) - dimensions
5 print(arr.ndim)      # 2 - number of dimensions
6 print(arr.size)      # 6 - total elements
7 print(arr.dtype)     # int64 - data type
8 print(arr.itemsize) # 8 - bytes per element
```

Slide 92: Data Types in NumPy

Category	Data Types
Integers	int8, int16, int32, int64
Unsigned	uint8, uint16, uint32, uint64
Floats	float16, float32, float64
Complex	complex64, complex128
Boolean	bool

Slide 93: Creating Arrays from Lists

```
# 1D array
arr1 = np.array([1, 2, 3])

# 2D array
arr2 = np.array([[1, 2], [3, 4]])

# Specify data type
arr3 = np.array([1, 2, 3], dtype=float)
# array([1., 2., 3.])
```

Slide 94: Array Creation Functions

```
np.zeros((3, 4))      # 3x4 array of zeros
np.ones((2, 3))       # 2x3 array of ones
np.full((3, 3), 7)    # 3x3 array of 7s
np.eye(4)             # 4x4 identity matrix
np.empty((2, 2))      # Uninitialized (garbage values)
```

Slide 95: Range Arrays

```
# arange([start,] stop[, step])
np.arange(10)         # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
np.arange(2, 10, 2)   # [2, 4, 6, 8]

# linspace(start, stop, num)
np.linspace(0, 1, 5) # [0, 0.25, 0.5, 0.75, 1.0]
```

Slide 96: Array Shape

```
arr = np.arange(12)  # shape (12,)
reshaped = arr.reshape(2, 6)  # shape (2, 6)

# Original unchanged
print(arr.shape)      # (12,)
print(reshaped.shape)# (2, 6)
```

Info

Total elements must match when reshaping!

Slide 97: Reshaping with -1

```
arr = np.arange(12)

# -1 means "infer this dimension"
reshaped = arr.reshape(-1, 3)  # (4, 3)
reshaped = arr.reshape(2, -1)  # (2, 6)
reshaped = arr.reshape(-1)     # (12,) - flatten
```

Info

Use -1 when you don't want to calculate one dimension.

Slide 98: Flattening Arrays

```
arr = np.array([[1, 2], [3, 4]])

# flatten() - returns a copy
flat_copy = arr.flatten()
flat_copy[0] = 99  # arr unchanged

# ravel() - returns a view (if possible)
flat_view = arr.ravel()
flat_view[0] = 99  # arr also changes!
```

Slide 99: Indexing 1D Arrays

```
arr = np.array([10, 20, 30, 40, 50])

# Single element access
arr[0]    # 10
arr[-1]   # 50

# Slicing (returns a view!)
slice_arr = arr[1:4]  # [20, 30, 40]
slice_arr[0] = 99     # Changes original arr!
```

Slide 100: Indexing 2D Arrays

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# arr[row, column]
arr[0, 1]     # 2 (first row, second column)
arr[1]        # [4, 5, 6] (second row, all columns)
arr[-1, -1]   # 9 (last row, last column)
```

Slide 101: Slicing 2D Arrays

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9,10,11,12]])

# rows 0-1, columns 1-2
arr[0:2, 1:3]  # [[2, 3], [6, 7]]

# All rows, column 0
arr[:, 0]      # [1, 5, 9]

# Row 1, all columns
arr[1, :]      # [5, 6, 7, 8]
```

Slide 102: 3D Array Indexing

```
# 3D array: (blocks, rows, columns)
arr_3d = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])

# arr[block, row, column]
arr_3d[0, 0, 0]  # 1
arr_3d[1, 1, 1]  # 8

# Use cases:
# - Images: (height, width, channels)
# - Time series: (time, x, y)
```

Slide 103: View vs Copy

View

- Shares data with original
- Created by slicing
- Memory efficient
- Modifying view changes original

Copy

- Independent copy of data
- Created with `.copy()`
- Uses more memory
- Modifications isolated

Slide 104: Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50, 60])

# Integer array indexing
arr[[0, 2, 4]]  # [10, 30, 50]

# 2D fancy indexing
arr2d = np.array([[1, 2], [3, 4], [5, 6]])
arr2d[[0, 2], [0, 1]]  # [1, 6] (positions (0,0) and (2,1))
```

Slide 105: Boolean Indexing

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Boolean mask
mask = arr > 5
# [False, False, False, False, False, True, True, True, True,
    True]

# Apply mask (returns 1D array)
arr[mask]  # [6, 7, 8, 9, 10]

# Direct filtering
arr[arr > 5]  # [6, 7, 8, 9, 10]
```

Slide 106: Combining Boolean Conditions

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# AND: use & (not 'and')
arr[(arr > 5) & (arr < 9)]  # [6, 7, 8]

# OR: use | (not 'or')
arr[(arr < 3) | (arr > 8)]  # [1, 2, 9, 10]

# NOT: use ~
arr[~(arr > 5)]  # [1, 2, 3, 4, 5]
```

Slide 107: where() Function

```
arr = np.array([-2, -1, 0, 1, 2, 3])

# np.where(condition, x, y)
# Returns x where True, y where False

# Clamp negative values to 0
result = np.where(arr > 0, arr, 0)
# [0, 0, 0, 1, 2, 3]

# Replace with different values
result = np.where(arr > 0, 'positive', 'non-positive')
```

Slide 108: Transposing Arrays

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

# Transpose (rows become columns)
arr.T
# [[1, 4],
#  [2, 5],
#  [3, 6]]

# Alternative
arr.transpose()  # Same as arr.T
```

Slide 109: Stacking Arrays (Vertical)

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# vstack = vertical stack (adds rows)
result = np.vstack((arr1, arr2))
# [[1, 2],
#  [3, 4],
#  [5, 6],
#  [7, 8]]

# Columns must match!
```

Slide 110: Stacking Arrays (Horizontal)

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# hstack = horizontal stack (adds columns)
result = np.hstack((arr1, arr2))
# [[1, 2, 5, 6],
#  [3, 4, 7, 8]]

# Rows must match!
```

Slide 111: Splitting Arrays

```
arr = np.arange(12)

# Split into 3 equal parts
np.split(arr, 3)  # [0-3], [4-7], [8-11]

# 2D splits
arr2d = np.arange(16).reshape(4, 4)
np.vsplit(arr2d, 2)  # Split vertically (2x4, 2x4)
np.hsplit(arr2d, 2)  # Split horizontally (4x2, 4x2)
```

Slide 112: Adding New Dimensions

```
arr = np.array([1, 2, 3])  # shape (3,)

# Add new axis (convert to column vector)
col_vec = arr[:, np.newaxis]  # shape (3, 1)
# [[1], [2], [3]]

# Add new axis (convert to row vector)
row_vec = arr[np.newaxis, :]  # shape (1, 3)
# [[1, 2, 3]]

# None also works (alias for np.newaxis)
col_vec = arr[:, None]
```

Slide 113: Removing Singleton Dimensions

```
arr = np.array([[[1], [2], [3]]])  # shape (1, 3, 1)

# squeeze() removes dimensions of size 1
squeezed = np.squeeze(arr)  # shape (3,)
# [1, 2, 3]

# Only removes dimensions where size = 1
arr2 = np.array([[1, 2, 3]])  # shape (1, 3)
squeezed2 = np.squeeze(arr2)  # shape (3,)
```

Slide 114: Random Number Generation

```
# Set seed for reproducibility
np.random.seed(42)

# Uniform [0, 1)
uniform = np.random.rand(3, 4)

# Standard normal (mean=0, std=1)
normal = np.random.randn(100)

# Random integers
integers = np.random.randint(0, 10, size=5)

# Custom uniform
uniform_custom = np.random.uniform(10, 20, size=10)
```

Slide 115: Code Example – Basic Array Operations

```python
import numpy as np

arr = np.arange(12).reshape(3, 4)
print(arr)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

print("Shape:", arr.shape)       # (3, 4)
print("Mean:", arr.mean())       # 5.5
print("Greater than 5:", arr[arr > 5])
# [ 6  7  8  9 10 11]
```

Slide 116: Element-wise Operations

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise (NOT matrix multiplication)
a + b    # [5, 7, 9]
a - b    # [-3, -3, -3]
a * b    # [4, 10, 18]
a / b    # [0.25, 0.4, 0.5]
```

Slide 117: Scalar Operations (Broadcasting)

```
arr = np.array([1, 2, 3, 4])

# Scalar applied to all elements
arr + 10    # [11, 12, 13, 14]
arr * 2     # [2, 4, 6, 8]
arr ** 2    # [1, 4, 9, 16]
np.sqrt(arr)  # [1, 1.41, 1.73, 2]
```

Slide 118: Arithmetic Functions

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.add(a, b)       # [5, 7, 9]
np.subtract(a, b) # [-3, -3, -3]
np.multiply(a, b) # [4, 10, 18]
np.divide(a, b)    # [0.25, 0.4, 0.5]
```

Slide 119: Matrix Multiplication

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication (3 ways)
result1 = np.dot(A, B)
result2 = A @ B       # Python 3.5+
result3 = A.dot(B)

# All produce: [[19, 22], [43, 50]]
```

Slide 120: Matrix Multiplication Rules

Dimension Requirements

- $(m \times n)$ @ $(n \times p) \rightarrow (m \times p)$
- $(3, 4)$ @ $(4, 2) \rightarrow (3, 2)$
- Inner dimensions must match!

Info

1D arrays are treated as row or column vectors as needed.

Slide 121: What is Broadcasting?

Definition

- Performing operations on arrays of different shapes
- NumPy automatically "stretches" the smaller array
- No actual memory copying (very efficient)
- Powerful and concise code

Slide 122: Broadcasting Rules

1. If dimensions differ, prepend 1s to the smaller shape
2. Compare dimensions from trailing to leading
3. Dimensions are compatible if:
   - They are equal, OR
   - One dimension is 1

Slide 123: Broadcasting Example 1

```
arr = np.ones((3, 4))  # shape (3, 4)
scalar = 10            # shape ()

# Scalar shape () -> (1, 1) after prepending
# Compare: (3, 4) vs (1, 1)
# 4 vs 1: OK, 3 vs 1: OK
result = arr + 10      # shape (3, 4) - works!
```

Slide 124: Broadcasting Example 2

```
arr = np.ones((3, 4))       # shape (3, 4)
col = np.array([1, 2, 3, 4])  # shape (4,)

# col shape (4,) -> (1, 4) after prepending
# Compare: (3, 4) vs (1, 4)
# 4 vs 4: OK, 3 vs 1: OK
result = arr + col          # shape (3, 4) - works!
```

Slide 125: Broadcasting Example 3

```
arr = np.ones((3, 4))       # shape (3, 4)
row = np.array([1, 2, 3])   # shape (3,)

# row shape (3,) -> (1, 3) after prepending?
# Wait, need to match trailing dimensions
# Better: reshape to (3, 1)
row_col = row.reshape(3, 1)  # shape (3, 1)

# (3, 4) vs (3, 1)
# 4 vs 1: OK, 3 vs 3: OK
result = arr + row_col     # shape (3, 4) - works!
```

Slide 126: Broadcasting Example 4 (Invalid)

Incompatible Shapes

- `arr.shape = (3, 4)`
- `other.shape = (2, 4)`
- Compare: 3 vs 2 $\rightarrow$ not equal, neither is 1
- **Error: cannot broadcast**

Slide 127: Broadcasting Visualized

Adding (3,1) to (3,4)

- The (3,1) array is repeated horizontally 4 times
- No actual copying, just logical stretching
- Memory efficient!

Slide 128: Broadcasting Benefits

Advantages

- No explicit Python loops
- Memory efficient (no actual copying)
- Concise, readable code

Info

Example: Center data: `data - data.mean(axis=0)`

Slide 129: Common Broadcasting Operations

```
# Standardization
normalized = (data - mean) / std

# Add bias to weights
weighted = weights + bias

# Outer product
outer = col[:, np.newaxis] * row

# Distance matrix
distances = np.sqrt((x[:, np.newaxis] - x)**2)
```

Slide 130: Code Example – Broadcasting

```
1  import numpy as np
2
3  # 100 samples, 10 features
4  data = np.random.rand(100, 10)
5
6  mean = data.mean(axis=0)  # shape (10,)
7  std = data.std(axis=0)    # shape (10,)
8
9  # Broadcasting in action!
10 normalized = (data - mean) / std
11
12 # Add column vector to matrix
13 matrix = np.ones((5, 5))
14 col_vec = np.arange(5).reshape(5, 1)
15 result = matrix + col_vec
```

Slide 131: What are Universal Functions (ufuncs)?

Ufuncs

- Functions that operate element-wise on arrays
- Vectorized wrappers for compiled C functions
- Very fast (no Python loops)

Slide 132: Math Ufuncs

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.add(a, b)       # [5, 7, 9]
np.subtract(a, b) # [-3, -3, -3]
np.multiply(a, b) # [4, 10, 18]
np.divide(a, b)    # [0.25, 0.4, 0.5]
np.power(a, 2)     # [1, 4, 9]
np.abs([-1, -2])   # [1, 2]
```

Slide 133: Trigonometric Ufuncs

```
angles = np.array([0, np.pi/2, np.pi])

np.sin(angles)     # [0, 1, 0]
np.cos(angles)     # [1, 0, -1]
np.tan(angles)     # [0, inf, 0]

np.arcsin([0, 1]) # [0, pi/2]
np.arccos([0, 1]) # [pi/2, 0]
np.arctan([0, 1]) # [0, pi/4]
```

Slide 134: Exponential and Log Ufuncs

```
arr = np.array([1, 2, 3])

np.exp(arr)     # e^1, e^2, e^3
np.exp2(arr)    # 2^1, 2^2, 2^3
np.log(arr)     # natural log
np.log10(arr)   # base 10
np.log2(arr)    # base 2
```

Slide 135: Other Useful Ufuncs

```
arr = np.array([-2.7, -1.2, 0.5, 1.8, 3.4])

np.sqrt(np.abs(arr))  # square root
np.square(arr)        # square
np.sign(arr)          # [-1, -1, 1, 1, 1]

np.ceil(arr)   # [-2, -1, 1, 2, 4]
np.floor(arr)  # [-3, -2, 0, 1, 3]
np.round(arr)  # [-3, -1, 0, 2, 3]

# Clip values to range
np.clip(arr, -1, 2)  # [-1, -1, 0.5, 1.8, 2]
```

Slide 136: Comparison Ufuncs

```
a = np.array([1, 2, 3])
b = np.array([3, 2, 1])

np.greater(a, b)    # [False, False, True]
np.less(a, b)       # [True, False, False]
np.equal(a, b)      # [False, True, False]

np.maximum(a, b)    # [3, 2, 3]
np.minimum(a, b)    # [1, 2, 1]
```

Slide 137: Logical Ufuncs

```
a = np.array([True, True, False, False])
b = np.array([True, False, True, False])

np.logical_and(a, b)  # [True, False, False, False]
np.logical_or(a, b)   # [True, True, True, False]
np.logical_not(a)     # [False, False, True, True]
np.logical_xor(a, b)  # [False, True, True, False]
```

Slide 138: Reductions (Aggregations)

```
arr = np.array([1, 2, 3, 4, 5])

np.sum(arr)     # 15
np.mean(arr)    # 3.0
np.std(arr)     # 1.414
np.var(arr)     # 2.0
np.min(arr)     # 1
np.max(arr)     # 5
```

Slide 139: Axis Parameter

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# axis=0: along rows (down columns)
arr.mean(axis=0)  # [4, 5, 6] (column means)

# axis=1: along columns (across rows)
arr.mean(axis=1)  # [2, 5, 8] (row means)

# No axis: entire array
arr.mean()        # 5.0
```

Slide 140: Keepdims Parameter

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

# Without keepdims
row_means = arr.mean(axis=1)  # shape (2,)

# With keepdims=True
row_means = arr.mean(axis=1, keepdims=True)  # shape (2, 1)
# [[2],
#  [5]]

# Useful for broadcasting!
normalized = arr - arr.mean(axis=1, keepdims=True)
```

Slide 141: Cumulative Operations

```
arr = np.array([1, 2, 3, 4, 5])

# Cumulative sum (running total)
np.cumsum(arr)  # [1, 3, 6, 10, 15]

# Cumulative product
np.cumprod(arr)  # [1, 2, 6, 24, 120]

# With axis
arr2d = np.array([[1, 2], [3, 4]])
np.cumsum(arr2d, axis=0)  # [[1, 2], [4, 6]]
np.cumsum(arr2d, axis=1)  # [[1, 3], [3, 7]]
```

Slide 142: NaN-Safe Aggregations

```
arr = np.array([1, 2, np.nan, 4, 5])

# Regular aggregations return NaN
np.mean(arr)  # nan

# NaN-safe aggregations ignore NaN
np.nanmean(arr)  # 3.0
np.nansum(arr)   # 12.0
np.nanstd(arr)   # 1.581
np.nanmin(arr)   # 1.0
np.nanmax(arr)   # 5.0
```

Slide 143: Other Aggregations

```
arr = np.array([1, 2, 3, 4, 5])

# Any / All
np.any([False, False, True])    # True
np.all([True, True, True])      # True

# Argmin / Argmax (indices)
np.argmin([3, 1, 4, 1, 5])      # 1
np.argmax([3, 1, 4, 1, 5])      # 4

# With axis
arr2d = np.array([[3, 1], [4, 2]])
np.argmin(arr2d, axis=0)  # [0, 0]
np.argmin(arr2d, axis=1)  # [1, 1]
```

Slide 144: Vectorization

Why Vectorize?

- Using ufuncs instead of Python loops
- **100x to 1000x faster**
- More readable code
- Less prone to bugs

Info

Example: `np.sin(arr)` instead of `[np.sin(x) for x in arr]`

Slide 145: Code Example – Ufuncs and Aggregations

```python
import numpy as np

arr = np.random.randn(1000, 50)

# Vectorized operations (no loops!)
arr_squared = arr ** 2
arr_centered = arr - arr.mean(axis=0)

# Aggregations
print("Mean:", arr.mean())
print("Std per column:", arr.std(axis=0))
print("Row sums:", arr.sum(axis=1))

# Cumulative sum
cumulative = np.cumsum(arr, axis=0)
```

Slide 146: Random Number Generation in Detail

```
# Set seed for reproducibility
np.random.seed(42)

# Uniform [0, 1)
uniform = np.random.rand(3, 4)

# Standard normal
normal = np.random.randn(1000)

# Random integers
integers = np.random.randint(0, 10, size=5)

# Custom uniform
uniform_custom = np.random.uniform(10, 20, 100)
```

Slide 147: Random Distributions

```
# Binomial: n trials, p success probability
binomial = np.random.binomial(n=10, p=0.5, size=100)

# Poisson: lambda = rate
poisson = np.random.poisson(lam=3, size=100)

# Uniform
uniform = np.random.uniform(low=0, high=1, size=100)

# Normal (Gaussian)
normal = np.random.normal(mu=50, sigma=10, size=100)
```

Slide 148: Statistical Analysis – Basic Stats

```
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

np.mean(data)          # 5.5 (average)
np.median(data)        # 5.5 (median)
np.percentile(data, 25)  # 3.25 (25th percentile)
np.var(data)           # 8.25 (variance)
np.std(data)           # 2.87 (standard deviation)
```

Slide 149: Correlation and Covariance

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Covariance matrix
cov_matrix = np.cov(x, y)
# [[2.5, 5.0],
#  [5.0, 10.0]]

# Correlation matrix
corr_matrix = np.corrcoef(x, y)
# [[1., 1.],
#  [1., 1.]]  (perfect positive correlation)
```

Slide 150: Sorting Arrays

```
arr = np.array([3, 1, 4, 1, 5, 9, 2])

# Returns sorted copy
sorted_arr = np.sort(arr)  # [1, 1, 2, 3, 4, 5, 9]

# Sort in-place
arr.sort()  # arr is now sorted

# Get indices that would sort
indices = np.argsort(arr)  # [1, 3, 6, 0, 2, 4, 5]

# 2D sorting
arr2d = np.array([[3, 1], [2, 4]])
np.sort(arr2d, axis=0)  # Sort columns
np.sort(arr2d, axis=1)  # Sort rows
```

Slide 151: Unique Values

```
arr = np.array([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])

# Unique values
unique = np.unique(arr)  # [1, 2, 3, 4]

# With counts
unique, counts = np.unique(arr, return_counts=True)
# unique: [1, 2, 3, 4]
# counts: [1, 2, 3, 4]

# With indices of first occurrences
unique, indices = np.unique(arr, return_index=True)
```

Slide 152: Histograms

```
data = np.random.randn(1000)

# 1D histogram
counts, bin_edges = np.histogram(data, bins=10)

# 2D histogram
x = np.random.randn(1000)
y = np.random.randn(1000)
counts2d, x_edges, y_edges = np.histogram2d(x, y, bins=20)

# Useful for probability distributions
prob = counts / counts.sum()
```

Slide 153: Masking and Filtering

```
arr = np.array([-3, -2, -1, 0, 1, 2, 3])

# Boolean mask
mask = arr > 0
# [False, False, False, False, True, True, True]

# Apply mask
positive = arr[mask]  # [1, 2, 3]

# Replace values in-place
arr[arr < 0] = 0  # [0, 0, 0, 0, 1, 2, 3]
```

Slide 154: Conditional Logic with where()

```
arr = np.array([-2, -1, 0, 1, 2])

# Simple where
result = np.where(arr > 0, arr, 0)
# [0, 0, 0, 1, 2]

# With scalars
result = np.where(arr > 0, 'positive', 'non-positive')

# With arrays
x = np.array([10, 20, 30])
y = np.array([100, 200, 300])
condition = np.array([True, False, True])
result = np.where(condition, x, y)  # [10, 200, 30]
```

Slide 155: Using where() for Multiple Conditions

```
arr = np.array([-2, -1, 0, 1, 2])

# Nested where (multiple conditions)
result = np.where(arr < 0, 'negative',
           np.where(arr == 0, 'zero', 'positive'))
# ['negative', 'negative', 'zero', 'positive', 'positive']

# Using select() for multiple conditions
conditions = [arr < 0, arr == 0, arr > 0]
choices = ['negative', 'zero', 'positive']
result = np.select(conditions, choices)
```

Slide 156: Polynomial Fitting

```
# Sample data
x = np.array([0, 1, 2, 3, 4])
y = np.array([0, 1, 4, 9, 16])  # y = x^2

# Fit polynomial of degree 2
coeffs = np.polyfit(x, y, deg=2)
# coeffs = [1, 0, 0]  (x^2 + 0x + 0)

# Evaluate polynomial
y_fit = np.polyval(coeffs, x)
# [0, 1, 4, 9, 16]

# Predict new values
y_pred = np.polyval(coeffs, [5, 6])  # [25, 36]
```

Slide 157: Linear Algebra in NumPy

```
A = np.array([[1, 2], [3, 4]])
b = np.array([5, 6])

# Matrix inverse
A_inv = np.linalg.inv(A)  # [[-2, 1], [1.5, -0.5]]

# Determinant
det = np.linalg.det(A)  # -2.0

# Eigenvalues and eigenvectors
eigenvals, eigenvecs = np.linalg.eig(A)

# Solve linear system Ax = b
x = np.linalg.solve(A, b)  # [-4, 4.5]
# Check: A @ x = b
```

Slide 158: Saving and Loading Arrays

```
arr = np.array([1, 2, 3, 4, 5])

# .npy format (binary, efficient)
np.save('array.npy', arr)
loaded = np.load('array.npy')

# .npz for multiple arrays
arr2 = np.array([10, 20, 30])
np.savez('arrays.npz', a=arr, b=arr2)
data = np.load('arrays.npz')
loaded_a = data['a']
loaded_b = data['b']
```

Slide 159: Text File I/O

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Save to CSV
np.savetxt('data.csv', arr, delimiter=',', fmt='%d')

# Load from CSV
loaded = np.loadtxt('data.csv', delimiter=',')

# With missing values
data_with_nan = np.genfromtxt('data.csv',
                              delimiter=',',
                              missing_values='NA',
                              filling_values=np.nan)
```

Info

Slide 160: Code Example – Data Analysis Workflow

```python
import numpy as np

# Generate synthetic data
np.random.seed(42)
data = np.random.normal(loc=50, scale=10, size=1000)

# Basic statistics
print(f"Mean: {data.mean():.2f}")
print(f"Median: {np.median(data):.2f}")
print(f"Std: {data.std():.2f}")

# Filter outliers (beyond 2 standard deviations)
mask = (data > data.mean() - 2*data.std()) & \
       (data < data.mean() + 2*data.std())
clean_data = data[mask]
print(f"Removed {len(data) - len(clean_data)} outliers")
```

End of Session 2

Questions?

Topics Covered:

- NumPy Arrays and Basics
- Broadcasting
- Universal Functions
- Data Analysis with NumPy